Laboratorio a coppie: gli errori più frequenti e il metodo

Lezione 16 — 1IOT, SST-ICT-01 — Prima sfida sulla piattaforma C·ARENA

09/11/2026

1 Che cosa abbiamo fatto oggi

Oggi abbiamo lavorato per tre ore **in autonomia**, a coppie, sulla piattaforma **C·ARENA**: un **pilota** scrive il codice, un **navigatore** prepara il piano e lo guida. Gli esercizi erano gli stessi della lezione 15: il **conto alla rovescia** (1B) e i **multipli fino a 100** con conteggio e somma (2B). Le soluzioni spiegate passo per passo sono nella **dispensa A15**.

La piattaforma **compila** il programma e lo prova con dei **test**: gli dà un numero come input e confronta quello che stampa con l'**uscita attesa**. Il test è verde solo se l'uscita è **quella giusta**, scritte comprese.

È andata così: quasi tutti i programmi erano **vicini** alla soluzione, ma pochi hanno avuto tutti i test verdi. Gli errori sono stati quasi sempre **gli stessi**, e quasi sempre di **metodo** più che di logica. In questa dispensa li trovi uno per uno, con la correzione e la regola da ricordare. Alla fine c'è il **metodo** per la prossima volta e una **lista di controllo** da usare prima di lanciare i test.

2 Come si legge un test che non passa

Quando un test non passa, la piattaforma mostra due cose: l'**uscita attesa** e l'**uscita del programma**. Il lavoro è **confrontarle riga per riga**, come in un gioco delle differenze.

Esempio

Test «parto da 5», input 5. Uscita attesa:

```
5 4 3 2 1
  Via!
```

Uscita del programma:

```
5 4 3 2 1
  via!
```

L'unica differenza è la **v minuscola**. Il programma è giusto al 99%, ma il test è rosso.

I **messaggi di richiesta** (`Inserisci un numero...`) sono liberi: il test non li guarda. Le **scritte del risultato** invece devono essere **identiche** all'esempio della consegna: stesse maiuscole, stessi due punti, stessi spazi.

Attenzione

Consegnare con 0 test superati non serve. Non dà punti e fa perdere la possibilità di correggere. Si consegna quando i test sono verdi, oppure quando si è fatto tutto il possibile e si è capito che cosa manca.

3 Errori di compilazione: leggere il messaggio

L'errore più costoso di oggi è stato **non leggere il messaggio del compilatore**. Il messaggio dice **la riga** e **che cosa non va**: va letto, non ignorato.

```c
#include <stdio.h>

int main()
{
  int n,x
  do {
```

Qui manca il punto e virgola dopo `int n,x`, alla **riga 5**. Il compilatore della piattaforma se ne accorge solo quando legge la riga dopo, quindi l'errore può indicare la **riga 6**: è normale. Regola: **guarda la riga indicata e quella prima**.

Il messaggio è in inglese, ma basta riconoscere poche parole:

Nel messaggio c'è…	Che cosa vuol dire e dove guardare
`';'`	manca un punto e virgola: guarda la fine della riga indicata e di **quella prima**
`undeclared`	usi una variabile che **non esiste** in quel punto: non l'hai dichiarata, oppure l'hai dichiarata dentro il `for` e la usi fuori
`'}'`	manca una **graffa chiusa**: conta le `{` e le `}`
`expression`	manca un pezzo: per esempio una condizione scritta a metà, come `mese < 1 || > 12`

Attenzione

Rilanciare lo stesso codice non cambia niente. Se premi «Lancia i test» senza aver modificato il programma, il risultato è identico: stesso errore, stessa riga. Prima **leggi**, poi **correggi**, poi **rilanci**.

3.1 Le graffe e l'indentazione

Ogni `{` ha la sua `}`. Se il codice è **indentato** (spostato a destra dentro ogni blocco), si vede subito quale graffa manca. Se è tutto attaccato al margine sinistro, trovarla è quasi impossibile.

```c
for (int i = n; i <= 100; i += n)
{
    printf("%d ", i);
    somma += i;
    conteggio++;
}
printf("\nMultipli fino a 100: %d\n", conteggio);
```

Anche la variabile `i` si capisce meglio così: è dichiarata **nel for**, quindi esiste **solo tra le sue graffe**. Dopo la graffa chiusa non c'è più: per questo il conteggio va in una variabile a parte, dichiarata prima del ciclo.

4 Errori nella lettura del numero

4.1 La & dentro le virgolette

```c
scanf("%d, &n");
```

Il programma compila, ma **non funziona**: `&n` è finito **dentro le virgolette**, quindi lo scanf non sa dove mettere il numero. Spesso il programma si blocca o non stampa niente. Forma giusta: le virgolette chiudono **subito dopo** `%d`, poi la virgola, poi `&n`.

```c
scanf("%d", &n);
```

4.2 La condizione del do while

Il `do while` **ripete la domanda** finché il numero è **sbagliato**. La condizione quindi descrive il numero sbagliato, con **gli stessi limiti della consegna**.

Consegna	Sbagliato	Giusto
da 1 a 20	`while (n < 1 || n < 20);`	`while (n < 1 || n > 20);`
da 2 a 9	`while (n < 2 || n > 20);`	`while (n < 2 || n > 9);`

Nel primo caso il programma rifiuta quasi tutti i numeri buoni. Nel secondo accetta anche il 10, che la consegna vieta: per questo c'è il test «numeri sbagliati prima», che dà prima numeri fuori dai limiti e controlla che il programma li **richieda**.

5 Errori nel ciclo

Prima di scrivere un ciclo bisogna rispondere a **tre domande**: **da dove parto?** **Quando mi fermo?** **Che cosa cambia a ogni giro?** Quasi tutti gli errori nel ciclo vengono da una risposta sbagliata o mancante.

5.1 Il ciclo che non parte mai

```
for (int a = numero; a >= numero * 10; a--)
{
    printf("%d ", a);
}
```

Con `numero` uguale a 5, la condizione è `5 >= 50`: **falsa già al primo giro**. Il ciclo non fa nemmeno un giro e sullo schermo compare solo quello che c'è dopo. Le tre risposte del conto alla rovescia sono: parto dal **numero**, continuo **finché sono almeno a 1**, a ogni giro **scendo di uno**.

```
for (int a = numero; a >= 1; a--)
{
    printf("%d ", a);
}
```

5.2 Il ciclo che non finisce

Se la piattaforma dice che il programma **non finisce**, la condizione è **sempre vera**: la variabile non cambia, oppure cambia nella direzione sbagliata. Esempio: parto da `n`, voglio scendere fino a 1, ma scrivo `i++` invece di `i--`: `i` sale, e `i >= 1` resta vera per sempre.

5.3 La condizione sul valore, non sul numero di giri

Nei multipli fino a 100 il ciclo si ferma quando il **multiplo** supera 100, non dopo un numero fisso di giri.

```
for (int a = 2; a <= 9; a++)
{
    printf("%d ", n * a);
}
```

Con 7 questo ciclo stampa da 14 a 63: **manca il 7** (si parte da 2 invece che da 1) e **si ferma troppo presto** (63 invece di 98). Il ciclo giusto parte dal numero stesso, sale di `n` e si ferma quando supera 100:

```c
for (int i = n; i <= 100; i += n)
{
    printf("%d ", i);
}
```

Attenzione anche al passo: si scrive `i += n`, non `i = i += n`.

6 Errori nella stampa

Questi sono gli errori più frustranti: il programma **calcola bene**, ma il test è rosso perché **stampa in modo diverso**.

6.1 Numeri attaccati o uno per riga

Istruzione	Sullo schermo	Va bene?
`printf("%d", i);`	`714212835...`	no: numeri attaccati
`printf("%d\n", i);`	un numero per riga	no: la consegna li vuole sulla stessa riga
`printf("%d ", i);`	`7 14 21 28 ...`	sì

6.2 Le scritte devono essere identiche

Scritto nel programma	Richiesto dal test
`via!`	`Via!`
`multipli fino a 100:  14`	`Multipli fino a 100:  14`
`somma 735`	`Somma:  735`

La regola è semplice: le scritte del risultato si **copiano dall'esempio** della consegna, lettera per lettera, **subito**, quando si scrive il printf. Sistemarle alla fine, una prova dopo l'altra, fa perdere molto tempo.

6.3 La riga che manca

Spesso il programma era giusto **tranne l'ultima riga**: mancava `Via!` dopo il conto alla rovescia, oppure mancavano le righe del conteggio e della somma. Il confronto riga per riga con l'uscita attesa lo mostra subito. `Via!` va **a capo** e **dopo il ciclo**, non dentro:

```c
for (int i = n; i >= 1; i--)
{
    printf("%d ", i);
}
printf("\nVia!\n");
```

7 Contatore e somma

Nella 2B, dopo i multipli, servono due righe in più: **quanti** sono e la loro **somma**. Servono due variabili:

- un **contatore**: parte da 0 e a ogni giro aumenta di 1 (`conteggio++`);
- un **accumulatore**: parte da 0 e a ogni giro aggiunge il numero (`somma += i`).

```c
int conteggio = 0;
int somma = 0;

for (int i = n; i <= 100; i += n)
{
    printf("%d ", i);
    conteggio++;
    somma += i;
}

printf("\nMultipli fino a 100: %d\n", conteggio);
printf("Somma: %d\n", somma);
```

```
7 14 21 28 35 42 49 56 63 70 77 84 91 98
Multipli fino a 100: 14
Somma: 735
```

Gli errori tipici sono tre:

- **non mettere a 0** contatore e somma: `int cicli;` senza `= 0` parte da un valore a caso, e il conteggio stampato è un numero assurdo (per esempio `-162758390`);
- **calcolarli ma non stamparli**: le variabili ci sono, ma senza i due printf dopo il ciclo le righe non compaiono;
- **stamparli dentro il ciclo**: escono a ogni giro invece che una volta sola alla fine.

8 Il metodo per la prossima volta

Gli errori di oggi si evitano quasi tutti con un **metodo**. Eccolo.

1. **Il piano prima del codice.** Il navigatore scrive le tre risposte: da dove parto, quando mi fermo, che cosa cambia a ogni giro. Il pilota non scrive finché il piano non c'è.
2. **Un pezzo alla volta, e subito la prova.** Prima solo la richiesta del numero con il do while: si lancia e si guarda. Poi il ciclo: si lancia e si guarda. Poi la riga finale. Poi contatore e somma. Non si scrive tutto il programma per poi scoprire venti errori insieme.

3. **Provare presto.** Anche un programma incompleto si può eseguire per vedere che cosa fa. Passare mezz'ora o un'ora senza nessuna prova vuol dire lavorare al buio.
4. **Una modifica alla volta.** Se cambio tre cose e il test è ancora rosso, non so quale delle tre era sbagliata.
5. **Leggere prima di rilanciare.** Errore di compilazione: leggo la riga e il messaggio. Test rosso: confronto l'uscita attesa con la mia, riga per riga.
6. **Le scritte dall'esempio.** Le copio subito, uguali.
7. **Gli aiutini servono**, ma tolgono il 10% dei punti: si usano dopo aver provato davvero.

Nota

Pilota e navigatore. Il **pilota** scrive e lancia i test. Il **navigatore** non sta a guardare: tiene il piano, controlla le scritte con l'esempio, legge ad alta voce i messaggi di errore e confronta le uscite. Due teste che guardano lo stesso programma trovano gli errori molto prima di una.

9 Lista di controllo: prima di premere «Lancia i test»

Prima di premere «Lancia i test»

- [ ] Ho **cambiato qualcosa** dall'ultima prova?
- [ ] Ogni istruzione finisce con il **punto e virgola**? Ogni `{` ha la sua `}`?
- [ ] Lo scanf è scritto `scanf("%d", &n);`, con la `&` fuori dalle virgolette?
- [ ] La condizione del do while usa **i limiti della consegna**?
- [ ] Il ciclo: **da dove parte**, **quando si ferma**, **che cosa cambia**? Ho controllato a mano il primo e l'ultimo giro?
- [ ] Nel printf dei numeri c'è lo **spazio** dopo `%d`?
- [ ] Le scritte sono **identiche all'esempio**, maiuscole e due punti compresi?
- [ ] C'è **l'ultima riga** (`Via!`, conteggio, somma), dopo il ciclo?
- [ ] Contatore e somma partono da **0**?